

# Recent advances in deep learning-based side-channel analysis

Stjepan Picek

A Workshop on TOPICS IN ADVANCED AREAS OF  
CRYPTOLOGY, 14.01.2026

# Outline

- 1 Deep Learning-based Side-channel Analysis
- 2 Common Directions
- 3 Recent Trends
- 4 Conclusions

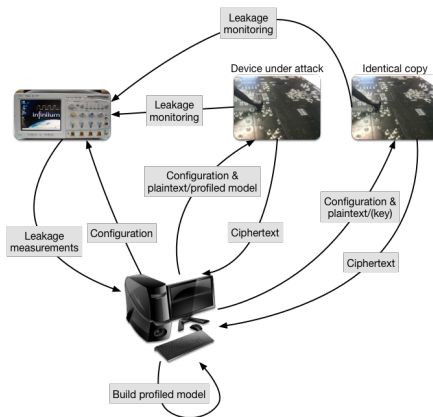
# Outline

- 1 Deep Learning-based Side-channel Analysis
- 2 Common Directions
- 3 Recent Trends
- 4 Conclusions

# Profiling Attacks

- Profiling attacks have a prominent place as the most powerful among side-channel attacks.
- Within profiling phase the adversary estimates leakage models for targeted intermediate computations, which are then exploited to extract secret information in the actual attack phase.
- Template Attack (TA) is the most powerful attack from the information theoretic point of view.
- Some **deep learning** (DL) techniques also belong to the profiling attacks.

# Profiling Attacks



- Profiling attacks are more complicated than the direct attacks.
- The attacker must have a copy of the device to be attacked.

# Deep Learning

- Stacked neural networks, i.e., networks consisting of multiple layers.
- Layers are made of nodes.

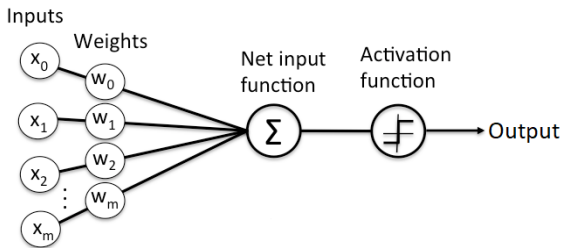
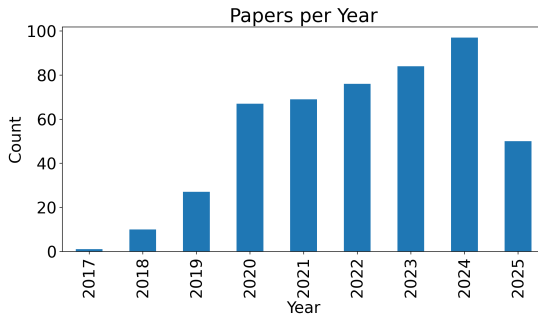


Figure: Perceptron.

# DLSCA



# Outline

- 1 Deep Learning-based Side-channel Analysis
- 2 Common Directions**
- 3 Recent Trends
- 4 Conclusions

# Hyperparameter Tuning

- Hyperparameter tuning is extremely important.
- Different algorithms have different hyperparameters.
- Neural networks have many hyperparameters.
- Random search.
- Grid search.
- Advanced techniques.
- Methodologies.

# Side-channel Analysis and Grammatical Evolution

- A neuroevolution framework that could replace random search to efficiently generate SCA models.



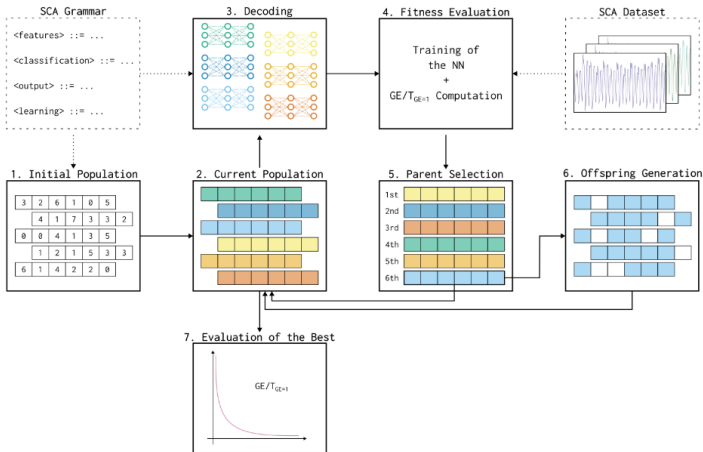
**Figure 1:** An example of GE decoding, using a grammar for evolving expressions.

---

```

<features> ::= <convolution> | <pooling>
<convolution> ::= layer:conv1d [num-filters ,int ,1,4,512]
<pooling> ::= <pool-type> [kernel-size ,int ,1,2,20] [stride ,int ,1,2,20]
<pool-type> ::= layer:pool-avg1d | layer:pool-max1d
...
<classification> ::= <fully-connected> | <dropout>
...
<learning> ::= <rmsprop> <early-stop> [batch_size ,int ,1,50,500]
              | ...
...
  
```

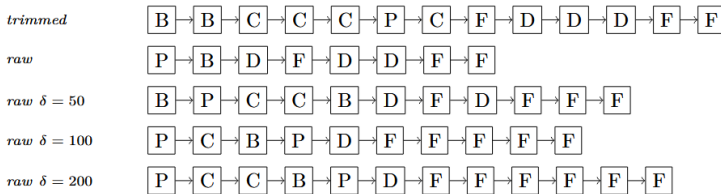
# Side-channel Analysis and Grammatical Evolution



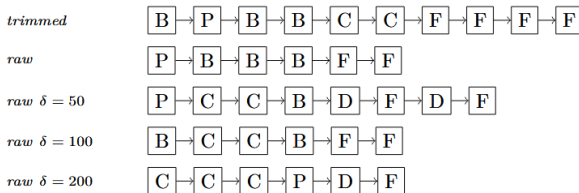
# Side-channel Analysis and Grammatical Evolution

| Dataset                         | $T_{GE=1}$ |    |         |         |         |        |
|---------------------------------|------------|----|---------|---------|---------|--------|
|                                 | SCAGE      | RS | [PWP22] | [HCM23] | [AGF22] | [LP24] |
| <i>trimmed</i>                  | 23         | 85 | 78      | -       | 120     | -      |
| <i>raw</i> $\delta_{max} = 0$   | 1          | 1  | 1       | 5       | -       | -      |
| <i>raw</i> $\delta_{max} = 50$  | 1          | 2  | -       | -       | -       | 33     |
| <i>raw</i> $\delta_{max} = 100$ | 1          | x  | 73      | 5       | -       | 251    |
| <i>raw</i> $\delta_{max} = 200$ | 1          | x  | -       | 4       | -       | 44     |

# Side-channel Analysis and Grammatical Evolution



(a) ASCADf NN architectures



(b) ASCADr NN architectures

# Public-key Crypto and SCA

- To mitigate side-channel attacks, real-world implementations of public-key cryptosystems adopt state-of-the-art countermeasures based on randomizing private or ephemeral keys.
- Usually, for each private key operation, a “scalar blinding” is performed using 32 or 64 randomly generated bits.
- Nevertheless, horizontal attacks based on a single trace still pose serious threats to protected ECC or RSA implementations.
- If the secrets learned through a single-trace attack contain too many wrong (or noisy) bits, the cryptanalysis methods for recovering the remaining bits become impractical due to time and computational constraints.

# Public-key Crypto and SCA

- By attacking several single traces, an attacker may recover several partially correct random private keys.
- This information is then used to label each sub-trace (trace interval representing the processing of a single private key bit) and use them as elements in a training set to train a neural network.
- Assuming that each recovered private key contains more than 50% of correct bits (just above a random guess), the trained neural network can significantly improve the number of correct bits in a random private key related to a single trace.
- The target is protected ECC implementations in software (protected  $\mu$ NaCl).

# Deep Learning-based Iterative Framework

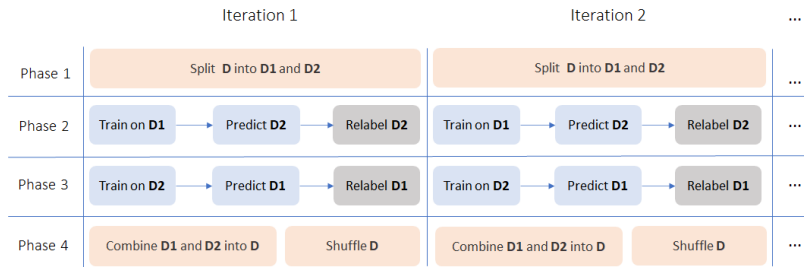
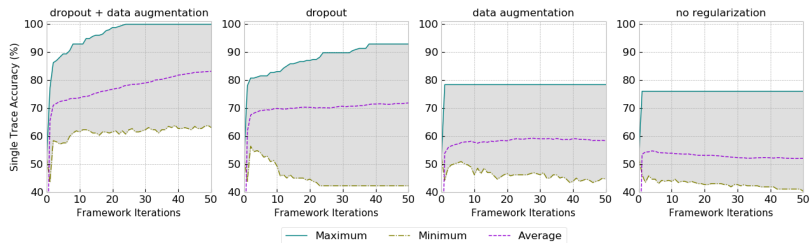


Figure: Proposed iterative framework

# Deep Learning-based Iterative Framework

- The procedure continues iteratively until a successful attack is achieved.
- In every step of this iterative process, it is expected that the amount of noisy labels decreases as a result of deep neural networks learning side-channel leakages from the limited correct labels in the training set.
- The higher the error bits in the initial training set, the more iterations we expect to need to reach a successful attack.

# Deep Learning-based Iterative Framework



**Figure:** Minimum, maximum, and average single trace accuracy with iterative framework on *cswap-arith* dataset.

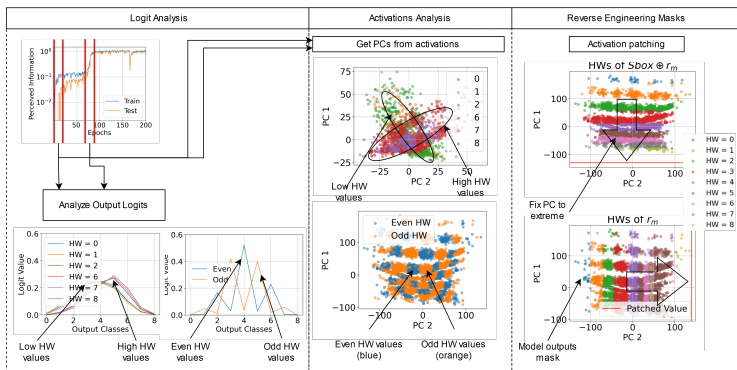
# Outline

- 1 Deep Learning-based Side-channel Analysis
- 2 Common Directions
- 3 Recent Trends**
- 4 Conclusions

# Open Directions

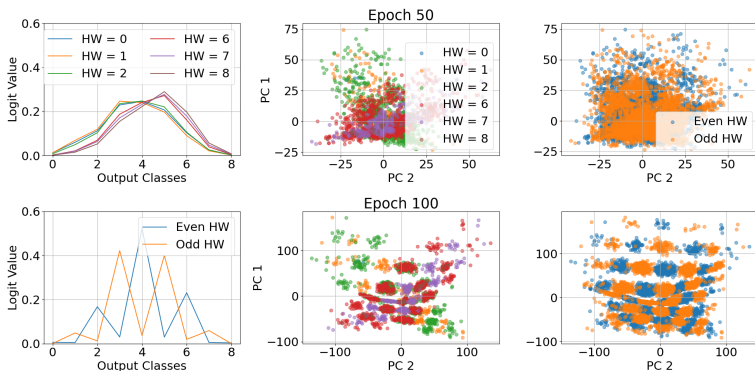
- Explainability.
- Different threat models.
- Unsupervised (or something like that) modes of operation.
- ...

# Mechanistic Interpretability



**Figure:** The analysis approach used in this study broadly consists of three major steps. After the performance increases are located using the PI metric, we plot logits to extract relevant features. Using these features, we plot the PCs of the activations and find the structure related to the leakage. Finally, we apply activation patching to reverse-engineer the masks. **Goal: understand the model's behavior and identify what information it extracts from the traces to make predictions.**

# Mechanistic Interpretability



**Figure:** Logit analysis (first column) and activation analysis (remaining columns) from models at epoch 50 (top) and epoch 100 (bottom) for CHES\_CTF. Legends for activation analysis are shared within columns. The difference in the number of points between the last two columns is due to not plotting the points for classes (HWs) 3, 4, and 5.

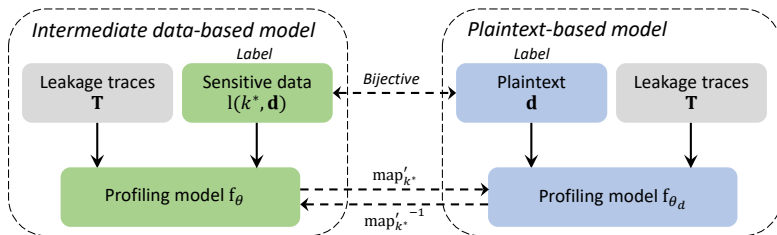
# Plaintext/Ciphertext-based Non-profiling SCA

- Supervised deep learning-based SCA learns a mapping based on known plaintexts and keys.
- Then, the adversary estimates the conditional probability given a leakage trace with the unknown key.
- In unsupervised setting, we do not know the key.
- But, the key is commonly fixed for all traces.
- The label  $l(k, d_i)$  and  $d_i$  would satisfy:

$$d_i \mapsto l(k, d_i). \quad (1)$$

$$l(k, d_i) = \text{map}_k(d_i), \quad (2)$$

# Plaintext/Ciphertext-based Non-profiling SCA



**Figure:** The relationship between intermediate data-based model and plaintext-based model.

## Plaintext/Ciphertext-based Non-profiling SCA

- For supervised DLSCA, if a profiling model is generalized well on the leakage traces, the probability of the incorrect value is closely correlated with the correct label.
- In unsupervised setting, we can still estimate the label distance, providing us with plaintext/ciphertext distribution.

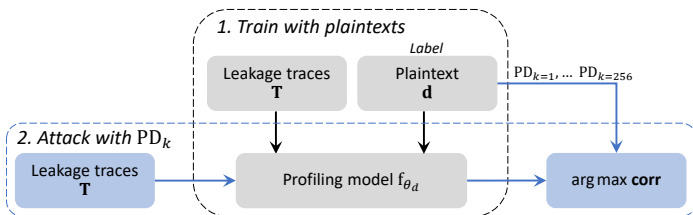


Figure: Attack scheme of the weakly supervised DLSCA.

$$k^* = \arg \max_k \text{corr}(PD_k^M(\mathbf{d}), f_{\theta_d}(\mathbf{T})), \quad k \in \mathcal{K}. \quad (3)$$

# Plaintext/Ciphertext-based Non-profiling SCA

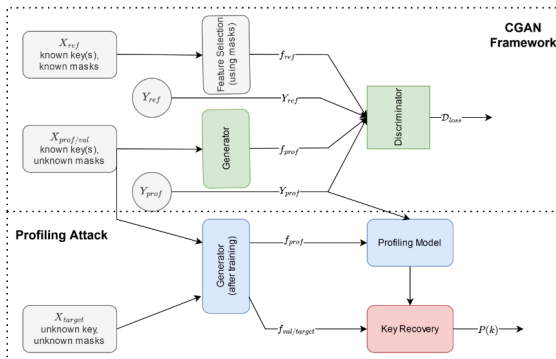
**Table:** Performance benchmark with non-profiling attacks.

| Dataset  | CPA         | MOR        | DDLA        | PLDL             |
|----------|-------------|------------|-------------|------------------|
| ASCAD_F  | KR161/KR47  | 1 957/638  | KR7/309     | <b>8/111</b>     |
| ASCAD_R  | KR64/KR8    | KR28/KR9   | 27 266/KR48 | <b>20/19</b>     |
| CHES_CTF | KR139/KR220 | KR6/KR31   | KR54/KR85   | <b>6 121/KR2</b> |
| AES_RD   | KR2/KR31    | KR33/3 112 | 2 541/KR2   | <b>1/57</b>      |
| AES_HD   | KR19/KR145  | 5 593/KR10 | KR26/KR20   | <b>60/KR6</b>    |

# From Black Box to White Box

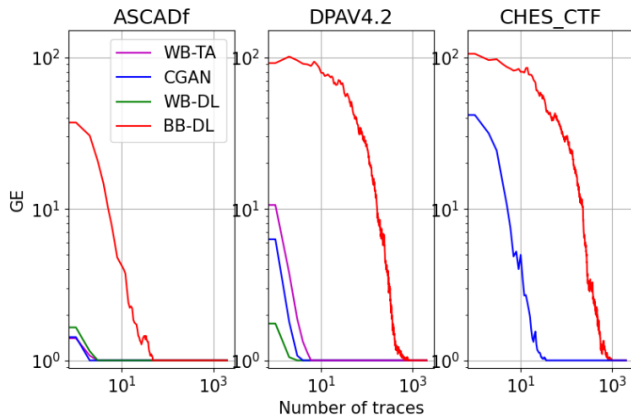
- The adversary possesses a similar implementation that can be used as a white-box reference design.
- We create an adversarial dataset by extracting features or points of interest from this reference design.
- These features are then utilized for training a conditional generative adversarial network (CGAN) framework, enabling a generative model to extract features from high-order leakages in protected implementation without any assumptions about the masking scheme or secret masks.

# CGAN Framework



Proposed CGAN-SCA framework.  $X_{ref}$  is the reference dataset with known secrets (i.e., masks and keys),  $X_{prof}$  is the profiling set with known key(s) and unknown masks, and  $X_{target}$  is the target device with unknown key and mask.  $Y_{ref}$  and  $Y_{prof}$  are corresponding labels to the  $X_{ref}$  and  $X_{prof}$  datasets, respectively.

# CGAN Profiling and Attack



GE results for key-byte 2 for various targets and methods (ref: ASCADr)

# Leakage Model-flexible DLSCA

- A deep learning approach with a flexible leakage model, referred to as the multi-bit model.
- Instead of trying to learn a pre-determined representation of the target intermediate data, we utilize the concept of the stochastic model to decompose the label into bits.
- Then, the deep learning model is used to classify each bit independently.
- This versatile multi-bit model can adjust to existing leakage models like the Hamming weight and Most Significant Bit while also possessing the flexibility to adapt to complex leakage scenarios.

# Leakage Model-flexible DLSCA

- Stochastic model approximates the linear portion of function to be learned using base functions but fails to encompass non-linear parts.
- Furthermore, it neglects potential multivariate key-dependent noise terms.
- These two constraints limit the discriminative power when identifying different leakages, leading to mediocre performance when, for instance, dealing with low numbers of side-channel traces

# Leakage Model-flexible DLSCA

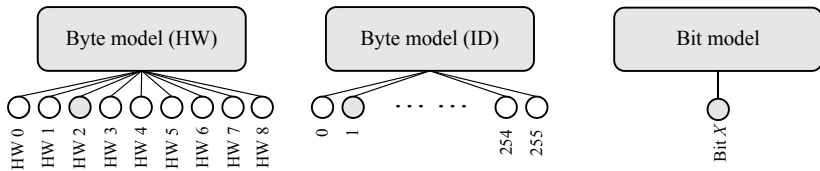


Figure: Conventional DLSCA models.

# Leakage Model-flexible DLSCA

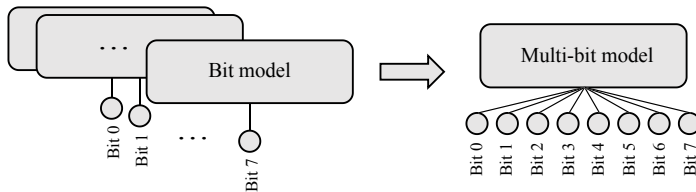


Figure: Multi-bit model.

# Leakage Model-flexible DLSCA

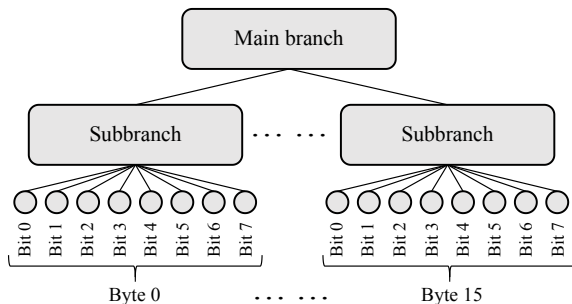


Figure: Multi-byte multi-bit DLSCA.

# Leakage Model-flexible DLSCA

$T_{GE0}$  of each subkey for the ASCAD\_F dataset.

|     | $k_0$ | $k_1$ | $k_2$ | $k_3$ | $k_4$ | $k_5$ | $k_6$ | $k_7$ | $k_8$ | $k_9$ | $k_{10}$ | $k_{11}$ | $k_{12}$ | $k_{13}$ | $k_{14}$ | $k_{15}$ |
|-----|-------|-------|-------|-------|-------|-------|-------|-------|-------|-------|----------|----------|----------|----------|----------|----------|
| MMB | 1     | 4     | 7     | 20    | 9     | 4     | 22    | 10    | 135   | 15    | 76       | 18       | 219      | 123      | 9        | 69       |
| MB  | 1     | 4     | 5     | 19    | 9     | 7     | 23    | 17    | 185   | 6     | 37       | 35       | 59       | 67       | 9        | 51       |
| HW  | 4     | 4     | 494   | 282   | 419   | 372   | 535   | 590   | x     | 628   | 1402     | 792      | x        | x        | 202      | x        |
| ID  | 2     | 2     | x     | 290   | 66    | 74    | 2415  | 111   | x     | 126   | 1272     | x        | x        | x        | 23       | x        |
| LSB | 17    | 17    | 1552  | 43    | 63    | 81    | 208   | 57    | 2683  | 49    | 138      | x        | x        | 1084     | 44       | 61       |
| MSB | 27    | 33    | x     | 1351  | 1079  | 983   | 984   | 866   | x     | 623   | x        | 517      | x        | x        | 3544     | x        |

Figure: Multi-byte multi-bit DLSCA Results.

# Collision-based DLSCA

- Side-channel collision attack (SCCA) is considered a non-profiling SCA (as it does not rely on a profiling device) but follows a different attack principle.
- It exploits data inter-dependence leaked during cryptographic procedures by targeting the collision of an internal state, which is more likely to coincide between two cryptographic operations.

# Collision-based DLSCA

- Concretely, an adversary monitors the side-channel information while the system processes different inputs and then searches for repeated leakage patterns signifying a collision event.
- When a collision is detected, the adversary uses this information to infer insights about the inter-dependencies of different key sections or the algorithm's internal state.

# Collision-based DLSCA

- For instance, let us consider the SubBytes operation of the Advanced Encryption Standard (AES) with the same substitution box (Sbox).
- The same data has been processed if two different Sbox operations lead to an identical side-channel pattern.

# Collision-based DLSCA

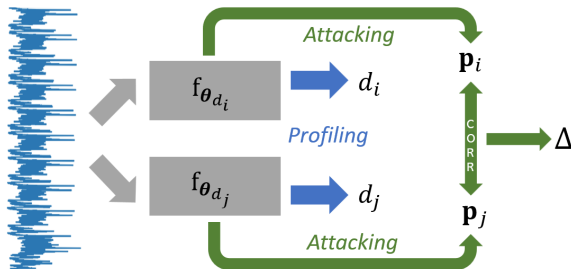


Figure: Plaintext-based correlation.

# Collision-based DLSCA

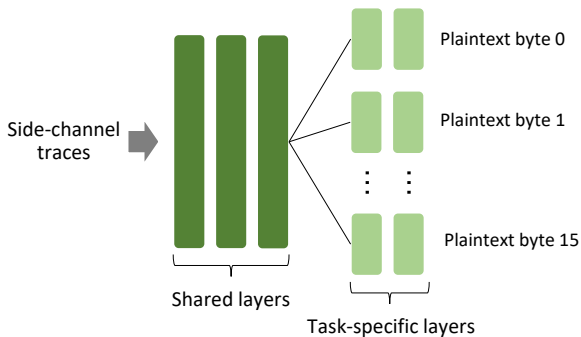


Figure: Deep learning architecture of plaintext-based SCCA.

## Collision-based DLSCA

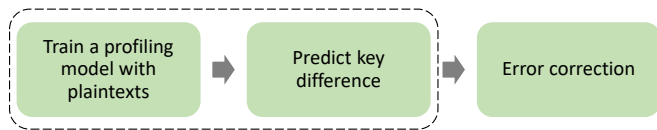


Figure: Plaintext-based SCCA.

# Collision-based DLSCA

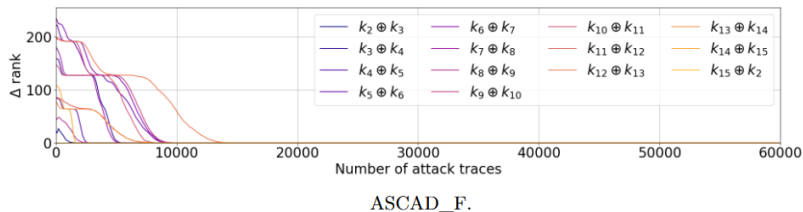


Figure: Example of results.

# Collision-based DLSCA

|           | Attack setting       | Samples                  | OSR       | Computation time |
|-----------|----------------------|--------------------------|-----------|------------------|
| DDLA      | Known leakage offset | Refined leakage interval | 100%      | 3h 14min         |
| DDLA      | Black box            | Full Set                 | 93%       | 5h 23min         |
| DL-SCCA   | Known leakage offset | Full Set                 | 0%/100%   | 23min            |
| DL-SCCA   | Black box            | Full Set                 | 0%/76%    | 3h 47min         |
| This work | Black box            | Full Set                 | 100%/100% | 5min             |

Figure: Performance comparison.

# Deep Learning-based Blind Side-channel Analysis

- In a blind SCA scenario, the attackers lack access to any known data (e.g., plaintext/ciphertext) and can only use the target device's side-channel measurements to deduce the key. This poses a significantly harder task than usual SCAs.
- Consists of three phases.

## Phase 1: Pre-computing theoretical joint distribution

- Suppose  $m$  is a public variable (plaintext or ciphertext) while  $y$  is the sensitive intermediate variable.
- Assuming that the leakage is following the Hamming weight ( $HW$ ) model, the joint distributions ( $HW(m), HW(y)$ ) of the public variable and the sensitive intermediate variables are distinct for every secret key  $k^*$ .
- In this phase, the theoretical joint distribution, denoted as  $(h_m^*, h_y^*)$ , is computed for all key candidates.
- This is done by iterating through all the keys and  $m$  to count the number of times  $(HW(m), HW(y))$  tuples appear.
- Then, we normalize the frequencies to obtain a probability distribution,  $Pr((h_m^*, h_y^*)|k)$ , for given hypothesis  $k$ .

## Phase 2: Labeling traces to obtain empirical distribution

- **Pols Selection:** To attain the empirical distribution, one first needs to identify a suitable Pol that represents  $HW(m)$  and another suitable Pol that represents  $HW(y)$ .
- We achieve this by assuming that the adversary uses correlation between the measurements and the Hamming weight of the targeted variable to locate the Pols.
- **Labeling Traces:** Next, one needs to obtain the empirical distribution using the selected Pols.
- This is achieved by labeling the traces and there are several options how to do that.

## Phase 3: Comparing the empirical distribution with the theoretical joint distribution

- Various methodologies were proposed to compare an empirical distribution with its theoretical joint distribution.
- The maximum likelihood criterion is a common (powerful) choice.

# Blind Side-channel Analysis

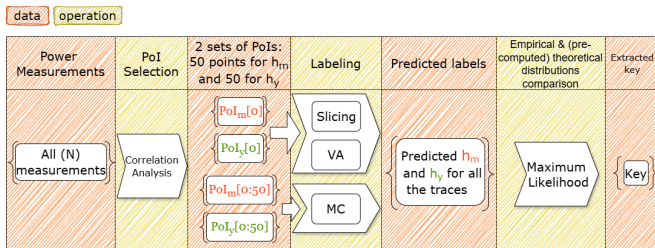


Figure: Blind Side-channel Analysis Framework.

# Deep Learning-based Blind Side-channel Analysis

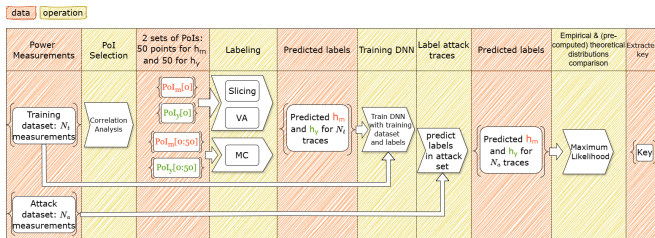


Figure: Deep Learning Blind Side-channel Analysis Framework.

# Deep Learning-based Blind Side-channel Analysis

- CRYSTAL-Kyber is the standard for Key Encapsulation Mechanism (KEM), which NIST selected for Post Quantum Cryptographic (PQC) applications.
- We focus on recovering the secret key from the *Decapsulation*( $sk, c$ ) procedure.

# Deep Learning-based Blind Side-channel Analysis

---

**Algorithm 1** Kyber.CPAPKE.Dec( $sk, c$ ): decryption

---

**Input:** Secret key  $sk \in \mathbb{B}^{12 \cdot k \cdot n/8}$

**Input:** Ciphertext  $c \in \mathbb{B}^{d_u \cdot k \cdot n/8 + d_v \cdot n/8}$

**Output:** Message  $m \in \mathbb{B}^{32}$

- 1:  $\mathbf{u} := \text{Decompress}_q(\text{Decode}_{d_u}(c), d_u)$
  - 2:  $\mathbf{v} := \text{Decompress}_q(\text{Decode}_{d_v}(c + d_u \cdot k \cdot n/8), d_v)$
  - 3:  $\hat{\mathbf{s}} := \text{Decode}_{12}(sk)$
  - 4:  $m := \text{Encode}_1(\text{Compress}_q(\mathbf{v} - \text{NTT}^{-1}(\hat{\mathbf{s}}^T \circ \text{NTT}(\mathbf{u})), 1))$
  - 5: **return**  $m$
- 

Figure: Attack point.

# Deep Learning-based Blind Side-channel Analysis

|                                   | Without DNN | CNN         | MLP         |
|-----------------------------------|-------------|-------------|-------------|
| <b>Linge et al. [53]</b>          | $GE = 1242$ | —           | —           |
| <b>Clavier &amp; Reynaud [13]</b> | $GE = 2415$ | —           | —           |
| <b>MC labeling</b>                | $GE = 1146$ | —           | —           |
| <b>DNN + Slicing</b>              | —           | $GE = 165$  | $GE = 255$  |
| <b>DNN + Slicing + Dropout</b>    | —           | $GE = 57$   | $GE = 58$   |
| <b>DNN + VA</b>                   | —           | $GE = 2369$ | $GE = 1372$ |
| <b>DNN + VA + Dropout</b>         | —           | $GE = 2356$ | $GE = 1143$ |
| <b>DNN + MC</b>                   | —           | $GE = 2.01$ | $GE = 9.2$  |
| <b>DNN + MC + Dropout</b>         | —           | $GE = 8.04$ | $GE = 16.2$ |

Figure: Attack results.

# Deep Learning-based Blind Side-channel Analysis

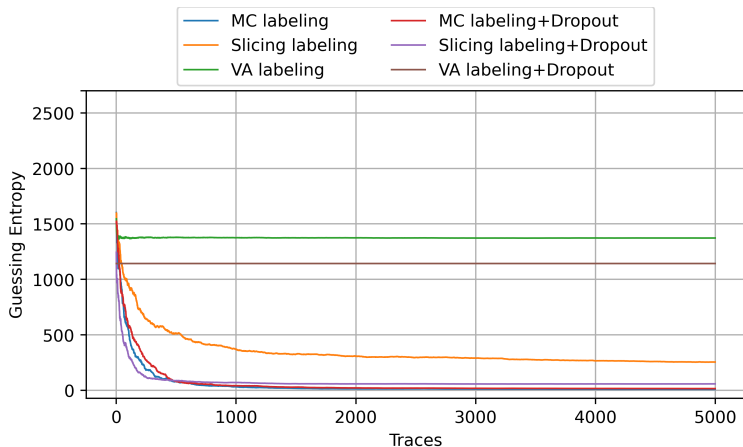


Figure: Guessing Entropy for the Kyber dataset using MLP.

# Outline

- 1 Deep Learning-based Side-channel Analysis
- 2 Common Directions
- 3 Recent Trends
- 4 Conclusions**

# Conclusions

- Deep learning-based SCA is rather active domain (and does not show signs of slowing down).
- As there are so many works available, it is challenging to recognize what and when to use.
- The results are very good but there are potential issues.
- The big challenges are unsupervised deep learning-based SCA (in certain flavors) and explainable AI for SCA.
- For PQC, the results are positive by scarce compared to, e.g., results for symmetric key crypto.
- Unclear do we want to validate “everything”.

# Challenges

- Saturation of the domain.
- New (ciphers, hardware, better countermeasures) targets.
- Lack of good datasets.
- Generative AI.
- “Just” an application of deep learning.
- Real-world applicability.
- ...

# Questions?

Thank you for your attention!

stjepan.picek@ru.nl